Universitatea Transilvania din Brașov

Facultatea de Inginerie Electrică și
Știința Calculatoarelor

Proiect la Arhitectura și Organizarea
Calculatoarelor (PAOC)

Sistem de alarmă cu detecție dublă Radar-PIR și alertare cu captură foto

Student:
Grigore Mihai-Alexandru
Grupa: 4LF721, Anul IV
mihai-alexandru.grigore@student.unitbv.ro

Cadru didactic îndrumător:
Prof. Dr. Ing. Carp
Marius Cătălin

Brașov

An universitar 2025-2026

Cuprins

Rezumat **3**

1 Introducere **4**
1.1 Contextul General și Motivație 4
1.2 Analiza Soluțiilor Existente și Abordarea Propusă 4
1.2.1 Limitările Senzorilor PIR 4
1.2.2 Avantajul Tehnologiei Radar mmWave 5
1.2.3 Abordarea Software: "Bare-Metal Programming" 5
1.3 Obiectivele Proiectului 5
1.4 Schema Bloc a Sistemului 6

2 Analiza Hardware și Arhitectura Sistemului **7**
2.1 Unitatea Centrală de Procesare: ESP32-S3 7
2.1.1 Organizarea Memoriei și Harta Regiștrilor (Memory Map) 8
2.2 Senzorii și Principiul de Funcționare 9
2.2.1 Radarul de Unde Milimetrice LD2410 (Tehnologie FMCW) 9
2.2.2 Senzorul PIR Digital AM312 10
2.3 Modulul de Cameră și Afișaj 10
2.3.1 Camera OV2640 10
2.3.2 LCD 1602 cu Interfață I2C Software 11
2.4 Schema Electrică a Sistemului 11
2.4.1 Lista de Conexiuni (Netlist) 12

3 Implementarea Software **13**
3.1 Mediul de Dezvoltare și Tehnologii Utilizate 13
3.1.1 Platforma de Dezvoltare (IDE) și Compilatoare 13
3.1.2 Limbaje de Programare Utilizate 13
3.1.3 Biblioteci și Dependențe Software 14
3.2 Arhitectura Modulară a Aplicației 14
3.3 Manipularea GPIO la Nivel de Registru 15
3.4 Driverul UART pentru Radar (Implementare Low-Level) 15
3.4.1 Accesul Direct la Cozile Hardware (FIFO) 15
3.4.2 Protocolul de Comunicație Hi-Link 17
3.4.3 Algoritmul de Parsare: Fereastra Glisantă 17

3.4.4 Decodarea Datelor și Endianness . . . 18
3.4.5 Configurarea Avansată a Sensibilității . . . 18
3.5 Driverul LCD I2C Software (Arhitectura Bit-Banging) . . . 19
3.5.1 Configurarea Electrică: Emularea Open-Drain . . . 19
3.5.2 Generarea Semnalelor I2C . . . 19
3.5.3 Interfațarea cu PCF8574 și HD44780 . . . 21
3.6 Logica de Automatizare și Fuziunea Senzorilor . . . 22
3.6.1 Algoritmul de Fuziune (Sensor Fusion) . . . 22
3.7 Gestiunea Camerei și Transmisia HTTP . . . 23
3.7.1 Achiziția Imaginii (Frame Buffer) . . . 23
3.7.2 Protocolul HTTP POST . . . 23
3.8 Arhitectura Backend (Server Python) . . . 24
3.8.1 API-ul REST . . . 24
3.8.2 Procesarea Fluxului Binar . . . 25
3.8.3 Diagrama Logică a Firmware-ului . . . 25

4 Rezultate Experimentale și Analiza Costurilor 28
4.1 Realizarea Practică a Sistemului . . . 28
4.2 Scenarii de Testare și Validare . . . 29
4.2.1 Scenariul A: Interferențe Termice (Doar PIR) . . . 29
4.2.2 Scenariul B: Mișcare prin Pereți (Doar Radar) . . . 29
4.2.3 Scenariul C: Intrus Real (Validare Duală) . . . 29
4.3 Interfața Web și Monitorizarea la Distanță . . . 30
4.4 Analiza Economică (Bill of Materials) . . . 31
4.5 Analiza Economică (Bill of Materials) . . . 31
4.5.1 Surse de Achiziție . . . 32

5 Concluzii și Direcții Viitoare 33
5.1 Bilanțul Realizărilor . . . 33
5.2 Dificultăți Întâmpinate și Soluții . . . 33
5.2.1 Incompatibilități Hardware (Footprint Radar) . . . 33
5.2.2 Sincronizarea Protocolului UART Proprietar . . . 34
5.3 Contribuții Personale . . . 34
5.4 Direcții de Dezvoltare Ulterioară . . . 34

A Listing-ul Integral al Codului Sursă 36
A.1 Configurarea Mediului (PlatformIO) . . . 36
A.2 Biblioteci Proprii (Drivere Low-Level) . . . 37
A.2.1 Accesul la Regiștrii GPIO . . . 37
A.2.2 Driverul Radar (UART FIFO) . . . 37
A.2.3 Driverul LCD (I2C Software) . . . 39
A.3 Firmware Principal (Bucla de Control) . . . 41
A.4 Script Server Backend (Python) . . . 46

Rezumat

Prezenta lucrare detaliază proiectarea și implementarea unui sistem de securitate hibrid, bazat pe microcontroller-ul ESP32-S3, care integrează tehnologia undelor milimetrice (Radar FMCW) cu detecția în infraroșu (PIR) pentru a elimina alarmele false specifice sistemelor convenționale.

Sistemul propus utilizează un algoritm de fuziune a datelor senzoriale (*Sensor Fusion*) implementat la nivel de firmware, validând prezența intrușilor doar prin confirmare dublă. Arhitectura software este dezvoltată preponderent prin acces direct la regiștrii hardware (Bare-Metal Programming), demonstrând optimizarea resurselor pentru periferice critice precum UART (pentru Radar) și GPIO (pentru protocolul I2C Software al afișajului LCD).

Componenta de alertare include captura vizuală utilizând senzorul OV2640 și transmisia imaginilor către un server backend dezvoltat în Python/Flask. Interfața utilizator este asigurată local printr-un afișaj LCD 1602 și de la distanță printr-un Dashboard Web accesibil în rețeaua locală.

Rezultatele experimentale confirmă robustețea soluției în fața interferențelor termice și capacitatea de a detecta subiecți statici prin monitorizarea micro-mișcărilor respiratorii, încadrându-se într-un buget de execuție redus (sub 150 RON). Proiectul evidențiază avantajele utilizării instrucțiunilor specifice arhitecturii Xtensa și a protocoalelor de comunicație optimizate pentru aplicații IoT de securitate.

Capitolul 1

Introducere

1.1 Contextul General și Motivație

În era digitalizării și a expansiunii conceptului Internet of Things (IoT), securitatea rezidențială și industrială a evoluat de la sisteme pasive, izolate, către ecosisteme interconectate capabile de decizii autonome. Statisticile recente indică o creștere exponențială a dispozitivelor smart-home, însă fiabilitatea acestora rămâne o provocare majoră.

Sistemele de alarmă convenționale, preponderente pe piață, se bazează în proporție covârșitoare pe senzori PIR (Passive Infrared). Deși sunt eficienți energetic și ieftini, acești senzori prezintă limitări fizice majore: sunt sensibili la curenți de convecție termică, nu pot detecta subiecți statici (care nu generează o variație de temperatură în spectrul IR) și sunt predispuși la alarme false cauzate de animale de companie sau surse de căldură variabile.

Pe de altă parte, sistemele de supraveghere video continuă necesită infrastructură complexă, lățime de bandă considerabilă și stocare masivă, fiind adesea invazive din punct de vedere al intimității (GDPR).

Proiectul de față propune o schimbare de paradigmă prin implementarea unui sistem hibrid de detecție. Prin fuziunea datelor de la un senzor Radar cu unde milimetrice (tehnologie FMCW - Frequency Modulated Continuous Wave) și un senzor PIR, sistemul elimină dezavantajele ambelor tehnologii, oferind o detecție robustă a prezenței umane, inclusiv a micro-mișcărilor (precum respirația), imposibil de detectat cu metodele clasice.

1.2 Analiza Soluțiilor Existente și Abordarea Propusă

Pentru a justifica arhitectura aleasă, este necesară o analiză a limitărilor tehnologiilor actuale comparativ cu soluția implementată:

1.2.1 Limitările Senzorilor PIR

Senzorii PIR funcționează prin detectarea variației radiației infraroșii emise de corpuri. Limitarea fundamentală este necesitatea mișcării transversale a subiectului. Un intrus

care se deplasează lent direct către senzor sau care rămâne imobil poate deveni "invizibil" pentru sistem.

1.2.2 Avantajul Tehnologiei Radar mmWave

Senzorul utilizat în acest proiect (LD2410, 24 GHz) utilizează efectul Doppler și modulația în frecvență pentru a măsura nu doar mișcarea, ci și distanța și energia țintei. Acesta poate penetra materiale nemetalice (plastic, lemn), permițând ascunderea senzorului pentru un design discret, și este imun la variațiile de lumină sau temperatură care afectează camerele și senzorii PIR.

1.2.3 Abordarea Software: "Bare-Metal Programming"

Un aspect distinctiv al acestui proiect este renunțarea voluntară la bibliotecile de nivel înalt (High-Level Libraries) specifice ecosistemului Arduino pentru perifericele critice. Deși bibliotecile standard abstractizează complexitatea hardware-ului, ele introduc adesea latențe și consumă resurse suplimentare (overhead).

Implementarea driverelor la nivel de registru (*Register-Level Programming*) pe arhitectura ESP32-S3 oferă avantaje clare:

- **Determinism:** Control precis asupra timing-ului semnalelor, esențial pentru protocolul I2C software (Bit-Banging) implementat.
- **Optimizare:** Reducerea dimensiunii codului compilat și a utilizării memoriei RAM.
- **Valoare Educațională:** Înțelegerea profundă a arhitecturii interne a SoC-ului (System on Chip), a hărții memoriei și a configurării matricei GPIO.

1.3 Obiectivele Proiectului

Scopul principal este dezvoltarea unui sistem integrat de securitate care să îndeplinească următoarele obiective specifice:

1. **Implementarea Detecției Duale (Sensor Fusion):** Integrarea hardware și software a senzorului Radar LD2410 și a senzorului PIR AM312. Sistemul trebuie să declanșeze alarma doar la validarea simultană a ambilor senzori, reducând rata alarmelor false spre zero.
2. **Dezvoltarea Driverelor Low-Level:** Scrierea de la zero a rutinelor de comunicare pentru:
    - **UART1:** Manipularea directă a bufferelor FIFO și a regiștrilor de stare pentru comunicarea de mare viteză (256.000 baud) cu radarul.
    - **I2C Software:** Implementarea protocolului de comunicație prin manipularea directă a pinilor GPIO (Set/Clear registers) pentru controlul afișajului LCD.

- **GPIO Matrix:** Configurarea manuală a multiplexorului de intrare/ieșire (IO_MUX) al procesorului ESP32-S3.

3. **Sistem de Alertare Vizuală și la Distanță:**
   - Captura de imagini utilizând modulul OV2640 la declanșarea evenimentului.
   - Transmisia securizată a imaginilor către un server HTTP centralizat (implementat în Python).
   - Feedback local prin interfață vizuală (LCD) și acustică (Buzzer).
4. **Conectivitate și IoT:** Integrarea stivei WiFi pentru controlul și monitorizarea stării sistemului (ARMAT/DEZARMAT) de la distanță, prin polling periodic către server.

1.4 Schema Bloc a Sistemului

Arhitectura hardware este centrată în jurul microcontroller-ului ESP32-S3, ales pentru capacitățile sale dual-core și suportul nativ pentru interfața camerei (DVP). Diagrama de mai jos ilustrează fluxul datelor și interconectarea modulelor periferice.

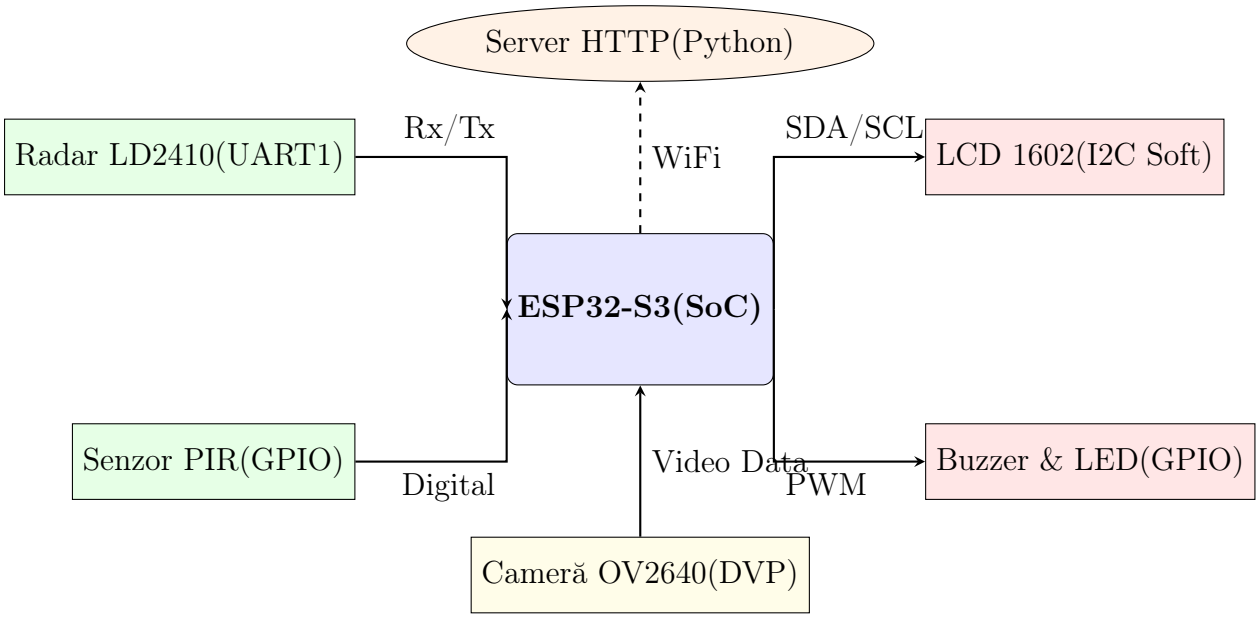


Figura 1.1: Schema Bloc a Sistemului de Alarmă

Sistemul funcționează pe baza unui algoritm de decizie distribuit: ESP32 preia datele brute de la senzori, le procesează local pentru a determina starea de alarmă și delegă stocarea dovezilor (imagini) către serverul extern, asigurând astfel o latență minimă în reacție.

Capitolul 2

Analiza Hardware și Arhitectura Sistemului

2.1 Unitatea Centrală de Procesare: ESP32-S3

Nucleul sistemului este reprezentat de System-on-Chip-ul (SoC) **ESP32-S3-WROOM-1**, o soluție de înaltă performanță produsă de Espressif Systems. Alegerea acestui model specific, în detrimentul variantelor clasice (ESP8266 sau ESP32-V1), a fost dictată de trei cerințe critice ale proiectului:

1. **Interfață Paralelă pentru Cameră (DVP):** ESP32-S3 include un controler hardware dedicat pentru interfața DVP (Digital Video Port) pe 8/16 biți, permițând transferul direct al fluxului video de la senzorul OV2640 în memoria RAM prin DMA (Direct Memory Access), fără a încărca procesorul.

2. **Vector Instructions (AI Acceleration):** Arhitectura Xtensa® LX7 Dual-Core la 240 MHz include instrucțiuni vectoriale care permit rularea eficientă a algoritmilor de procesare a semnalului (DSP), necesari pentru interpretarea datelor brute de la radar.

3. **Matrice GPIO Flexibilă:** Spre deosebire de alte microcontrollere unde funcțiile periferice sunt legate de pini ficși, ESP32-S3 dispune de o matrice GPIO complet configurabilă, permițând rutarea semnalelor UART și I2C către orice pin fizic, aspect exploatat în design-ul PCB-ului nostru.

Deoarece proiectul implică manipularea directă a pinilor prin matricea GPIO, cunoașterea distribuției fizice a acestora este esențială. Figura de mai jos ilustrează dispunerea pinilor pentru placa de dezvoltare utilizată.

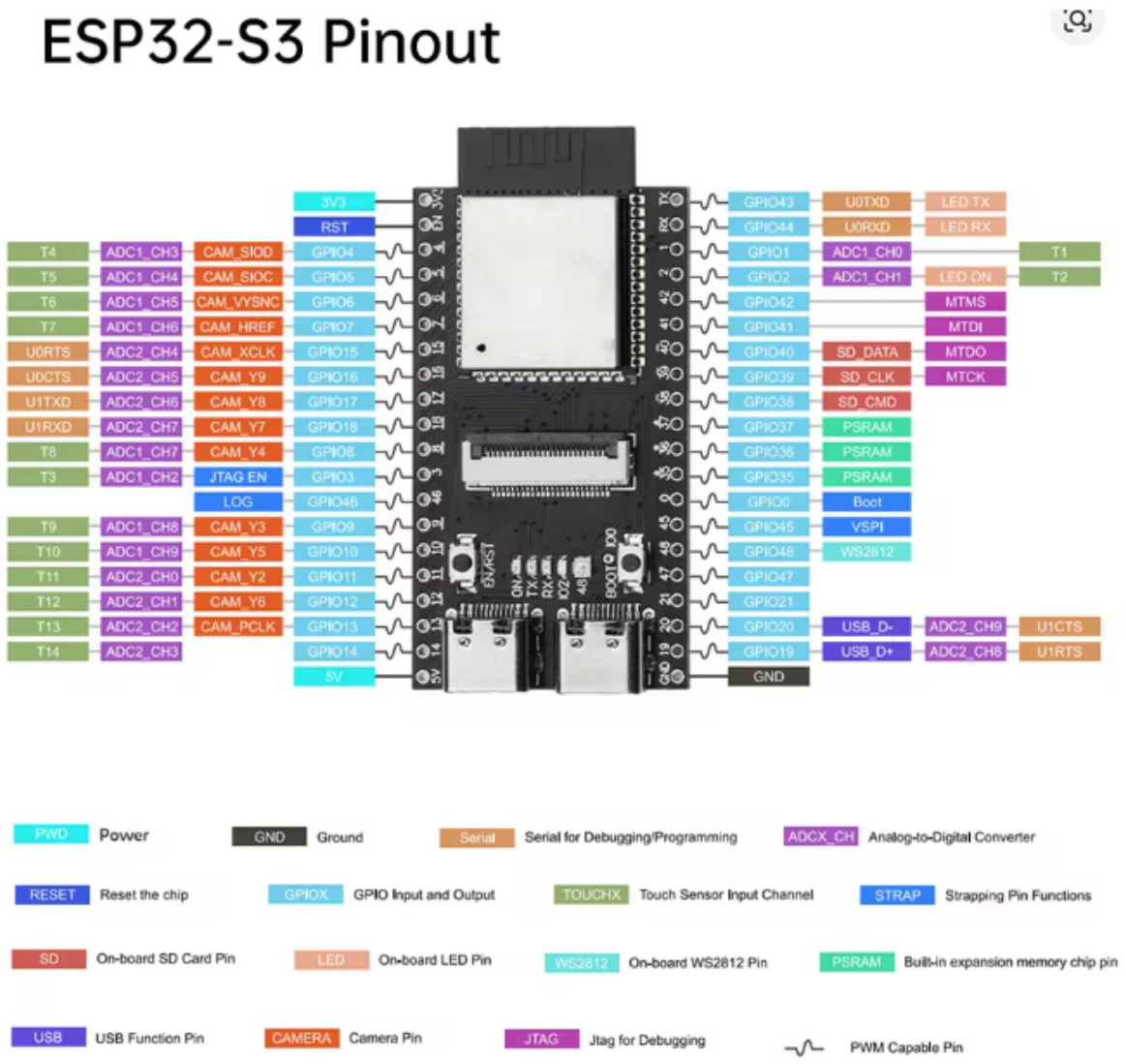


Figura 2.1: Diagrama pinilor (Pinout) pentru ESP32-S3 WROOM N16R8

2.1.1 Organizarea Memoriei și Harta Regiștrilor (Memory Map)

Pentru a obține performanțe maxime, driverele dezvoltate în acest proiect ocolesc straturile de abstracție (HAL) și interacționează direct cu hardware-ul. Acest lucru se realizează prin conceptul de *Memory-Mapped I/O*.

În arhitectura ESP32-S3, regiștrii de control ai perifericelor sunt mapați într-o zonă continuă de memorie virtuală, începând cu adresa de bază `0x6000 0000`. Tabelul de mai jos detaliază segmentele de memorie accesate de driverele `Radar_Driver.h` și `LCD_Driver.h`.

Periferic	Adresa de Bază	Funcție în Proiect
UART0	`0x6000 0000`	Debugging și Bootloader log
UART1	`0x6000 1000`	Comunicație High-Speed cu Radarul (FIFO Buffer)
GPIO	`0x6000 4000`	Control Buzzer, LED și citire PIR (Bit-Banging)
IO_MUX	`0x6000 9000`	Configurare electrică (Pull-up, Drive Strength)
SYSTEM	`0x600C 0000`	Gestiune Ceasuri (Clock Gating) și Reset

Tabela 2.1: Harta Memoriei pentru Perifericele Utilizate (ESP32-S3)

2.2 Senzorii și Principiul de Funcționare

2.2.1 Radarul de Unde Milimetrice LD2410 (Tehnologie FMCW)

Senzorul Hi-Link LD2410 reprezintă o evoluție majoră față de senzorii Doppler clasici (CW). Acesta utilizează tehnologia **FMCW (Frequency Modulated Continuous Wave)** în banda ISM de 24 GHz.

Principiul de funcționare se bazează pe emisia unui semnal a cărui frecvență variază liniar în timp (Chirp). Semnalul reflectat de țintă este mixat cu semnalul emis, iar diferența de frecvență rezultată (Beat Frequency) este direct proporțională cu distanța până la țintă. Mai mult, prin analiza spectru-ului Doppler, senzorul poate detecta mișcări micrometrice (ex: expansiunea toracelui în timpul respirației), permițând detecția persoanelor statice, o capacitate critică pentru un sistem de securitate modern.

Interfațarea cu ESP32 se realizează via UART la un Baud Rate atipic de **256.000 bps**, necesar pentru a transmite fluxul dens de date de telemetrie.

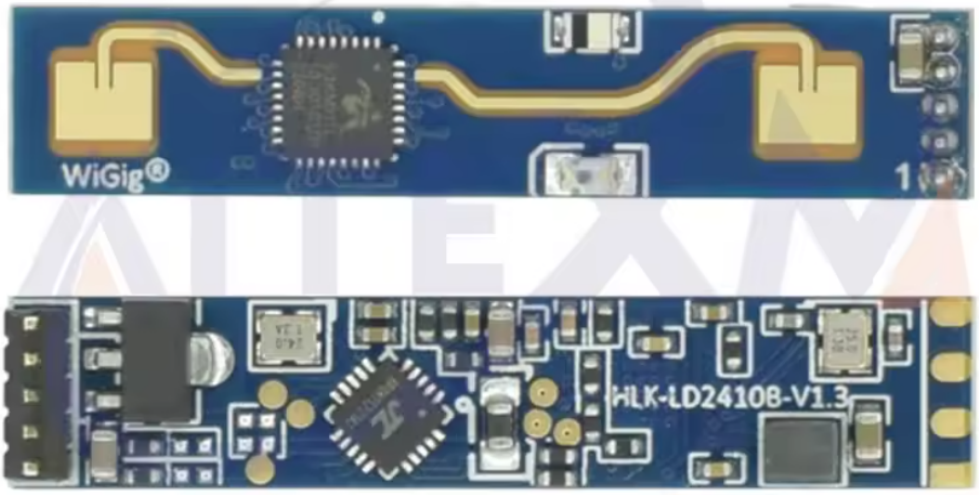


Figura 2.2: LD2410B, senzor radar

2.2.2 Senzorul PIR Digital AM312

Pentru detecția pasivă în infraroșu, am selectat senzorul **AM312**, o variantă miniaturizată și digitalizată a clasicului HC-SR501.

Alegerea AM312 aduce avantaje tehnice majore în contextul IoT:

- **Imunitate la Interferențe RF:** Spre deosebire de senzorii analogici ieftini, AM312 integrează procesarea semnalului (DSP) direct în capsulă, fiind mult mai rezistent la zgomotul electromagnetic generat de modulul WiFi al ESP32-S3.
- **Latență și Timp de Reținere:** Senzorul are un timp de "Delay" fix de 2 secunde și funcționează în mod *Repeatable Trigger*, menținând ieșirea HIGH atâta timp cât mișcarea persistă.
- **Consum Energetic:** Cu un curent de repaus de sub 20μA, este ideal pentru sisteme alimentate pe baterii.

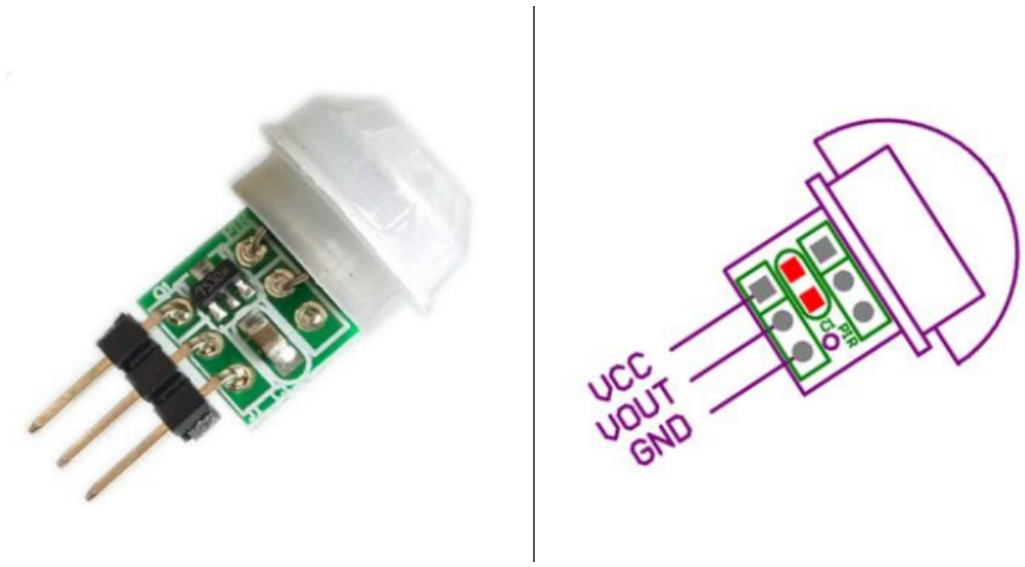


Figura 2.3: AM312, senzor PIR

2.3 Modulul de Cameră și Afișaj

2.3.1 Camera OV2640

Senzorul de imagine OV2640 este un SoC CMOS complet, capabil să realizeze compresie JPEG hardware în timp real. Aceasta este o caracteristică vitală, deoarece transmiterea unei imagini RAW (necomprimate) prin WiFi ar bloca procesorul. Configurarea senzorului se face prin protocolul SCCB (echivalent I2C), iar datele sunt transmise pe o magistrală paralelă de 8 biți (D0-D7), sincronizate de semnalele PCLK (Pixel Clock), VSYNC și HREF.

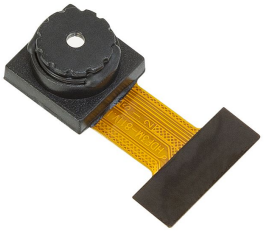
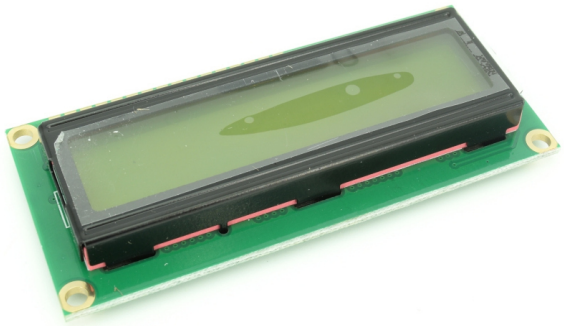


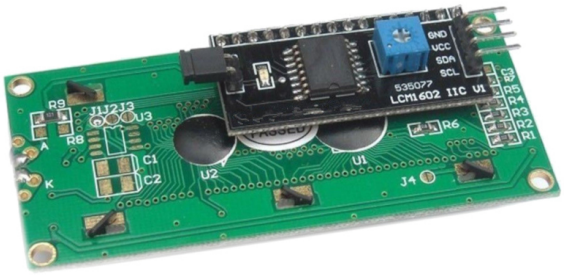
Figura 2.4: OV2640, camera

2.3.2 LCD 1602 cu Interfață I2C Software

Afișajul alfanumeric este controlat prin intermediul unui expander de I/O PCF8574T. Deoarece pinii hardware I2C erau alocați camerei în configurația inițială, am implementat un driver software. Acesta emulează protocolul I2C prin manipularea directă a regiștrilor `GPIO_OUT_W1TS` (Set) și `GPIO_OUT_W1TC` (Clear), generând manual semnalele de START, STOP și CLOCK, o tehnică cunoscută sub numele de *Bit-Banging*.



(a) Vedere frontală (Afișaj)



(b) Vedere posterioară (Adaptor I2C cu PCF8574)

Figura 2.5: Modulul LCD 1602 și adaptorul serial I2C utilizat

După cum se observă în Figura 2.5b, modulul adaptor (backpack-ul negru) conține cipul PCF8574 și un potențiometru albastru pentru reglarea contrastului. Comunicarea cu acest cip se realizează prin cele 4 fire vizibile în partea dreaptă (GND, VCC, SDA, SCL).

2.4 Schema Electrică a Sistemului

Interconectarea modulelor hardware este realizată conform tabelului de conexiuni de mai jos, având ca element central placa de dezvoltare ESP32-S3 WROOM N16R8. Alimentarea sistemului se face printr-o sursă stabilizată de 5V (USB-C), iar modulele peri-

ferice sunt alimentate din rail-ul de 5V al plăcii de dezvoltare, cu excepția liniilor de date care lucrează la nivel logic de 3.3V.

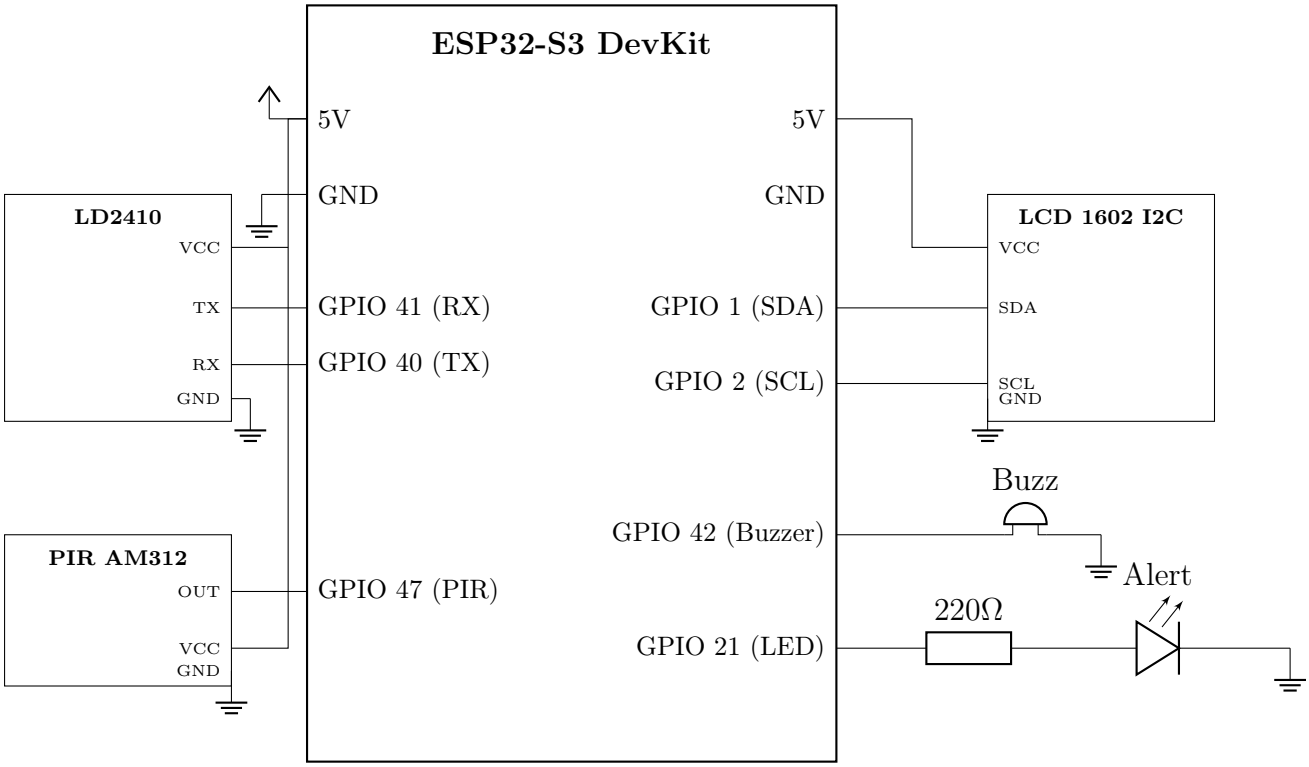


Figura 2.6: Schema Electrică Detaliată

2.4.1 Lista de Conexiuni (Netlist)

Pentru claritate, conexiunile fizice realizate sunt detaliate în tabelul următor. Se observă conexiunea încrucișată (Cross-Over) a liniilor UART pentru Radar și utilizarea pinilor I2C definiți software.

Componentă	**Pin Componentă**	**Pin ESP32-S3**	**Observații**
Radar LD2410	TX	GPIO 41	UART1 RX (Cross)
Radar LD2410	RX	GPIO 40	UART1 TX (Cross)
Radar LD2410	VCC	5V	Consum max 150mA
Senzor PIR AM312	OUT	GPIO 47	High=Mișcare (Delay 2s)
LCD 1602	SDA	GPIO 1	I2C Data (Open-Drain)
LCD 1602	SCL	GPIO 2	I2C Clock (Open-Drain)
Buzzer Activ	V+	GPIO 42	Activ pe High
LED Alertă	Anod (+)	GPIO 21	Activ pe High

Tabela 2.2: Tabelul de conexiuni (Pinout)

Capitolul 3

Implementarea Software

3.1 Mediul de Dezvoltare și Tehnologii Utilizate

Conform cerințelor de proiectare și pentru a asigura robustețea firmware-ului, procesul de dezvoltare a fost realizat utilizând un set de unelte software profesionale, configurate specific pentru arhitectura Xtensa a procesorului ESP32-S3.

3.1.1 Platforma de Dezvoltare (IDE) și Compilatoare

Deși ecosistemul Arduino oferă un IDE propriu, pentru acest proiect s-a optat pentru utilizarea **Visual Studio Code** (VS Code), echipat cu extensia **PlatformIO** sau **Arduino Extension**. Această alegere este motivată de facilitățile avansate oferite:

- **IntelliSense:** Autocompletare inteligentă și navigare rapidă prin definițiile registrilor hardware (esențial pentru programarea Low-Level).
- **Managementul Dependențelor:** Gestionarea automată a bibliotecilor și a versiunilor acestora.
- **Compilatorul:** S-a utilizat toolchain-ul `xtensa-esp32s3-elf-gcc` (bazat pe GCC), configurat cu flag-uri de optimizare `-O2` pentru a balansa dimensiunea codului cu viteza de execuție.

Sistemul de operare gazdă utilizat pentru dezvoltare a fost **Windows 10/11** (64-bit), asigurând compatibilitatea cu driverele USB-Serial CH340/CP210x necesare programării plăcii.

3.1.2 Limbaje de Programare Utilizate

Proiectul este poligot, îmbinând programarea de sistem cu programarea de aplicație:

- **C++ (Standard C++11/17):** Utilizat pentru firmware-ul ESP32. Alegerea C++ a permis utilizarea Programării Orientate pe Obiecte (OOP) pentru a encapsula driverele senzorilor (`class RadarLD2410`, `class LCD_I2C`) și pentru a gestiona memoria eficient prin pointeri.

- **Python 3.10:** Utilizat pentru dezvoltarea serverului backend. Python a fost ales pentru rapiditatea dezvoltării și suportul excelent pentru manipularea fișierelor și a socket-urilor de rețea.
- **HTML5 / CSS3:** Utilizate pentru randarea interfeței grafice (Dashboard) servite de aplicația Python, permițând vizualizarea datelor în orice browser modern.

3.1.3 Biblioteci și Dependențe Software

Pentru a accelera dezvoltarea, au fost integrate biblioteci software validate de comunitate, însă cu o selecție riguroasă pentru a nu introduce latențe. Tabelul 3.1 prezintă versiunile exacte utilizate, conform cerințelor de reproductibilitate[cite: 135].

Bibliotecă	**Versiune**	**Sursă/Autor**	**Rol în Proiect**
`Arduino Core for ESP32`	2.0.11	Espressif Systems	HAL de bază și Bootloader
`esp_camera.h`	2.0.4	Espressif Systems	Driver pentru senzorul OV2640
`WiFi.h`	Native	Espressif Systems	Stiva TCP/IP și modul WiFi
`HTTPClient.h`	Native	Espressif Systems	Client pentru cereri POST/GET
`ArduinoJson.h`	6.21.3	Benoit Blanchon	Parsare eficientă JSON
`Flask` (Python)	2.3.3	Pallets Projects	Micro-framework Web Server

Tabela 3.1: Inventarul bibliotecilor software și versiunile utilizate

Este important de menționat că pentru modulele critice (Radar și LCD), nu s-au folosit biblioteci externe, ci s-au dezvoltat drivere proprii (*Custom Drivers*) prezentate în secțiunile următoare, pentru a demonstra competențele de programare la nivel de registru.

3.2 Arhitectura Modulară a Aplicației

Dezvoltarea software a urmat o paradigmă hibridă, separând straturile de abstracție în funcție de criticitatea temporală a sarcinilor. Codul este organizat în module distincte (fișiere header `.h`), incluse în bucla principală `main.cpp`. Această structură facilitează depanarea și reutilizarea codului.

- **Nivelul Hardware (Low-Level):** Drivere personalizate pentru Radar (`Radar_Driver.h`), LCD (`LCD_Driver.h`) și GPIO (`GPIO_Registers.h`). Acestea accesează direct memoria mapată a perifericelor.
- **Nivelul Aplicație (Mid-Level):** Logica de fuziune a senzorilor și mașina de stări a sistemului de alarmă.
- **Nivelul Rețea (High-Level):** Comunicarea WiFi și protocolul HTTP pentru interacțiunea cu serverul Python.

3.3 Manipularea GPIO la Nivel de Registru

Un element central al optimizării este fișierul `GPIO_Registers.h`. Accesul standard `digitalWrite` din Arduino introduce un overhead semnificativ (câteva microsecunde) datorită verificărilor de siguranță.

Soluția implementată utilizează funcții `inline` marcate cu atributul `IRAM_ATTR` pentru a rula direct din memoria RAM a instrucțiunilor (Instruction RAM), eliminând latențele de acces la memoria Flash.

De asemenea, s-a tratat corect arhitectura ESP32-S3 care împarte pinii în două "bănci":

- **Bank 0:** Pinii 0-31 (Registrul `GPIO_OUT_W1TS_REG`).
- **Bank 1:** Pinii 32-53 (Registrul `GPIO_OUT1_W1TS_REG`).

```
// Macro pentru identificarea bancii de pini
#define IS_BANK_1(pin) (pin >= 32)
#define GET_BIT_POS(pin) (IS_BANK_1(pin) ? (pin - 32) : pin)

// Functie inline pentru setarea rapida a pinului (HIGH)
inline void IRAM_ATTR reg_write_high(int pin) {
    if (IS_BANK_1(pin))
        REG_WRITE(GPIO_OUT1_W1TS_REG, (1UL << GET_BIT_POS(pin)));
    else
        REG_WRITE(GPIO_OUT_W1TS_REG, (1UL << GET_BIT_POS(pin)));
}
```

Listing 3.1: Macro-uri optimizate pentru controlul GPIO

3.4 Driverul UART pentru Radar (Implementare Low-Level)

Senzorul LD2410 comunică printr-o interfață serială (UART) la o viteză de transmisie atipică de **256.000 baud**. Această viteză ridicată, combinată cu frecvența mare de eșantionare a radarului (10Hz pentru datele procesate), generează un flux continuu de date care poate satura rapid bufferele software standard ale bibliotecii Arduino.

Pentru a asigura o latență minimă și un determinism temporal strict, am renunțat la driverele HAL (Hardware Abstraction Layer) și am implementat accesul direct la controllerul UART1 al SoC-ului ESP32-S3.

3.4.1 Accesul Direct la Cozile Hardware (FIFO)

Microcontrollerul ESP32-S3 dispune de buffere circulare hardware (FIFO - First In, First Out) de 128 de bytes pentru fiecare periferic UART. Accesul la aceste buffere se face prin maparea memoriei (Memory Mapped I/O).

Citirea Datelor (RX Pipeline)

Operația de citire standard (`Serial.read()`) implică multiple verificări de eroare și copieri de memorie. Implementarea noastră accesează direct registrul `UART_FIFO_REG(1)`. Citirea de la această adresă returnează instantaneu cel mai vechi byte din coada hardware și decrementează pointerul intern de citire al perifericului.

Pentru a evita citirea din bufferul gol (Buffer Underflow), interogăm registrul de stare `UART_STATUS_REG(1)`.

```
// Definirea adreselor de baza (din esp32-s3 technical reference manual)
#define UART1_BASE 0x60001000

// Citirea directa din FIFO (fara overhead software)
uint8_t readReg() {
    // Accesam adresa 0x60001000 (FIFO)
    return (uint8_t)REG_READ(UART_FIFO_REG(1));
}

// Verificarea starii cozii RX
int availableReg() {
    // UART_STATUS_REG contine informatii despre TX si RX
    // Bitii [0:9] reprezinta numarul de bytes in RX FIFO (rx_fifo_cnt)
    // Masca 0x3FF (0011 1111 1111) izoleaza acesti 10 biti
    return (REG_READ(UART_STATUS_REG(1)) & 0x3FF);
}
```

Listing 3.2: Rutine optimizate pentru accesul la UART1

Scrierea Datelor (TX Pipeline)

Transmiterea comenzilor de configurare necesită scrierea în același registru FIFO. Totuși, trebuie monitorizat gradul de ocupare al cozii de transmisie pentru a evita suprascrierea datelor (Buffer Overflow). Biții [16:23] din registrul de stare indică `tx_fifo_cnt`.

```
void writeReg(const uint8_t* data, int len) {
    for (int i = 0; i < len; i++) {
        // Asteapta cat timp coada TX este aproape plina (>= 126 bytes)
        // Shiftare dreapta 16 biti pentru a aduce tx_fifo_cnt pe
    pozitia 0
        while (((REG_READ(UART_STATUS_REG(1)) >> 16) & 0xFF) >= 126);

        // Scrie byte-ul in hardware
        REG_WRITE(UART_FIFO_REG(1), data[i]);
    }
}
```

Listing 3.3: Scrierea blocantă în TX FIFO

3.4.2 Protocolul de Comunicație Hi-Link

Senzorul LD2410 utilizează un protocol binar proprietar, structurat în pachete de date (Frames). Structura unui pachet standard de "Engineering Mode" este următoarea:

Header (4B)	Len (2B)	Payload (N Bytes)	Footer (4B)
`F4 F3 F2 F1`	`N + 2`	Date Distanță / Energie	`F8 F7 F6 F5`

Tabela 3.2: Structura Pachetului de Date LD2410

3.4.3 Algoritmul de Parsare: Fereastra Glisantă

Deoarece UART-ul este un protocol asincron de tip stream (flux continuu), nu există o garanție că citirea începe la începutul unui pachet. Pentru a sincroniza fluxul de date, am implementat un algoritm de tip *Sliding Window* (Fereastră Glisantă).

Algoritmul menține un buffer de 4 bytes care se actualizează la fiecare nou byte primit (FIFO software). Când conținutul ferestrei coincide cu secvența magică `0xF4 0xF3 0xF2 0xF1`, sistemul intră în starea de captură a datelor utile.

```
void actualizeaza() {
    while (availableReg() > 0) {
        uint8_t b = readReg(); // Extrage urmatorul byte din hardware

        // STARE 1: Cautare Sincronizare (Header)
        if (!headerFound) {
            // Shiftare buffer fereastra glisanta
            buffer[0] = buffer[1];
            buffer[1] = buffer[2];
            buffer[2] = buffer[3];
            buffer[3] = b;

            // Verificare semnatura Header
            if (buffer[0] == 0xF4 && buffer[1] == 0xF3 &&
                buffer[2] == 0xF2 && buffer[3] == 0xF1) {
                headerFound = true; // Sincronizare reusita
                bufIndex = 0;       // Reset index payload
            }
        }
        // STARE 2: Captura Payload
        else {
            buffer[bufIndex++] = b;

            // Avem nevoie de primii 18 bytes relevanti dupa header
            if (bufIndex >= 18) {
                decodeData();      // Proceseaza datele
                headerFound = false; // Reset pentru urmatorul pachet
                bufIndex = 0;
            }
        }
```

```
    }
}
```

Listing 3.4: Implementarea Mașinii de Stări pentru Parsare

3.4.4 Decodarea Datelor și Endianness

Protocolul utilizează formatul **Little-Endian** pentru reprezentarea numerelor pe 16 biți (byte-ul cel mai puțin semnificativ este transmis primul). Reconstrucția valorilor de distanță necesită operații pe biți (bitwise operations).

- **Byte 4 (Target State):** Indică tipul detecției.
  - `0x00`: Nicio țintă.
  - `0x01`: Țintă în mișcare.
  - `0x02`: Țintă statică.
  - `0x03`: Ambele.
- **Bytes 5-6 (Move Distance):** Distanța în centimetri.
- **Bytes 8-9 (Static Distance):** Distanța în centimetri.

```
// Exemplu: Daca Buffer[5]=0xE8 si Buffer[6]=0x03
// 0x03E8 = 1000 cm
distMisc = buffer[5] | (buffer[6] << 8);
distStat = buffer[8] | (buffer[9] << 8);
stare = buffer[4];
```

Listing 3.5: Reconstrucția datelor din Buffer

3.4.5 Configurarea Avansată a Sensibilității

Pentru a elimina alarmele false cauzate de obiecte în mișcare irelevante (ventilatoare, perdele), radarul a fost reconfigurat dinamic la inițializare. Comanda trimisă modifică pragurile de sensibilitate (Gate Thresholds) pentru porțile de distanță apropiată.

Secvența hexazecimală transmisă este: `FD FC FB FA 14 00 64 00 00 00 02 00 ...`

- `FD FC FB FA`: Header-ul de configurare (diferit de cel de date).
- `14 00`: Lungimea pachetului (20 bytes).
- `64 00`: Codul comenzii (Set Sensitivity).
- `02 00`: Indexul Porții (Gate 2 - aprox 1.5m).
- `2D 00 00 00`: Valoarea sensibilității (45/100).

Această configurare asigură că radarul ignoră micro-vibrațiile, dar rămâne sensibil la respirația umană în raza de 4 metri.

3.5 Driverul LCD I2C Software (Arhitectura Bit-Banging)

Deși ESP32-S3 dispune de două controlere hardware I2C, utilizarea acestora impune restricții legate de maparea pinilor și de configurarea complexă a ceasurilor interne. Pentru a demonstra controlul total asupra semnalelor digitale și pentru a permite utilizarea oricărei perechi de pini GPIO, am implementat un driver I2C pur software, tehnică cunoscută sub numele de **Bit-Banging**.

Această abordare presupune generarea manuală a formelor de undă (ceas și date) prin scrierea directă în regiștrii de ieșire ai procesorului, respectând diagramele de timp specificate în standardul I2C (NXP Semiconductors UM10204).

3.5.1 Configurarea Electrică: Emularea Open-Drain

Protocolul I2C este definit pe o magistrală de tip *Open-Drain* (sau Open-Collector). Aceasta înseamnă că dispozitivele Master și Slave pot doar să "traga" linia la masă (Logic 0), dar nu o pot forța activ la 3.3V (Logic 1). Nivelul High este asigurat pasiv de rezistențele de Pull-Up externe.

Configurarea standard a unui pin GPIO este *Push-Pull* (totem-pole), ceea ce ar cauza scurtcircuite dacă două dispozitive ar încerca să scrie valori opuse simultan. Pentru a evita acest lucru, am manipulat registrul `GPIO_PIN_PAD_DRIVER`.

```
void configPin(int pin) {
    // 1. Configurare IO_MUX: Functie GPIO, Input Enable, Pull-Up
    volatile uint32_t* mux = (volatile uint32_t*)(0x60009000 + (pin * 4));
    *mux = (1 << 12) | (1 << 9) | (1 << 8);

    // 2. Activare Output Driver (GPIO_ENABLE)
    if (pin < 32) REG_WRITE(GPIO_ENABLE_W1TS_REG, (1 << pin));

    // 3. Activare Mod Open-Drain (Pad Driver = 1)
    // Bitul 2 din registrul pinului activeaza tranzistorul NMOS
    // dar dezactiveaza PMOS-ul de pe partea de High-Side.
    volatile uint32_t* pin_reg = (volatile uint32_t*)(0x60004088 + (pin * 4));
    *pin_reg |= (1 << 2);
}
```

Listing 3.6: Configurarea Open-Drain prin Registri

3.5.2 Generarea Semnalelor I2C

Protocolul I2C se bazează pe două linii: SDA (Serial Data) și SCL (Serial Clock). Stabilitatea datelor este critică: SDA se poate modifica doar atunci când SCL este LOW. Dacă SDA își schimbă starea în timp ce SCL este HIGH, acest lucru este interpretat ca o comandă specială (START sau STOP).

Condițiile de START și STOP

Funcția `i2c_start()` inițiază tranzacția prin coborârea liniei SDA, urmată de SCL, respectând timpul de setup (`t_SU;STA`) de minim 4.7 μs (emulat prin `delayMicroseconds`).

```
void i2c_start() {
    // Secventa: SDA High, SCL High -> SDA Low -> SCL Low
    sda_high();
    scl_high();
    delayMicroseconds(2); // t_buf
    sda_low();
    delayMicroseconds(2); // t_hd;sta
    scl_low();
}
```

Listing 3.7: Generarea manuală a secvenței START

Transmisia Byte-cu-Byte

Datele sunt transmise serial, MSB first (Most Significant Bit). Pentru fiecare bit, driverul execută următorii pași:

1. Setează SDA conform bitului curent (0 sau 1).
2. Așteaptă stabilizarea semnalului.
3. Ridică SCL (Clock High) - moment în care Slave-ul (LCD-ul) citește bitul.
4. Coboară SCL (Clock Low) - pregătirea următorului bit.

```
bool i2c_writeByte(uint8_t data) {
    for (int i = 0; i < 8; i++) {
        // Masca 0x80 verifica bitul cel mai semnificativ
        if (data & 0x80) sda_high(); else sda_low();
        data <<= 1;           // Shiftare stanga pentru urmatorul bit

        delayMicroseconds(1);
        scl_high();           // Puls ceas (Slave sample)
        delayMicroseconds(2);
        scl_low();            // Ceas jos (Setup next bit)
        delayMicroseconds(1);
    }
    // Generare puls ceas pentru ACK (ignoram citirea efectiva)
    sda_high(); scl_high(); delayMicroseconds(2); scl_low();
    return true;
}
```

Listing 3.8: Bucla de transmisie a unui octet

3.5.3 Interfațarea cu PCF8574 și HD44780

Ecranul LCD 1602 utilizează controlerul paralel HD44780. Pentru a-l conecta prin I2C, se utilizează un expander de I/O pe 8 biți, PCF8574. Byte-ul trimis prin I2C este mapat direct la pinii LCD-ului în următorul format:

$$D_7\ D_6\ D_5\ D_4\ BL\ EN\ RW\ RS$$

Unde:

- **D4-D7:** Cei 4 biți de date (High Nibble).
- **BL:** Backlight (1=On, 0=Off).
- **EN:** Enable (Semnal de ceas pentru LCD).
- **RS:** Register Select (0=Comandă, 1=Date).

Modul de Operare pe 4 biți

Deoarece PCF8574 oferă doar 8 linii, iar LCD-ul ar necesita 11 linii în mod complet (8 date + 3 control), driverul operează LCD-ul în modul "4-bit". Aceasta implică spargerea fiecărui byte de date în două jumătăți (Nibbles):

1. Se trimit cei 4 biți superiori (High Nibble).
2. Se generează un puls pe pinul EN (Enable) pentru a latche-a datele.
3. Se trimit cei 4 biți inferiori (Low Nibble).
4. Se generează din nou un puls pe EN.

```
void sendNibble(uint8_t nibble, uint8_t mode) {
    // Compunerea byte-ului pentru PCF8574
    // Pastram Backlight-ul activ si setam modul (Comanda/Date)
    uint8_t data = (nibble & 0xF0) | mode | _backlightVal;

    // Generare Puls Enable (Low -> High -> Low)
    writePCF8574(data);          // Setup
    writePCF8574(data | LCD_EN); // EN = 1 (Rising Edge)
    delayMicroseconds(1);
    writePCF8574(data & ~LCD_EN);// EN = 0 (Falling Edge - Data Latch)
    delayMicroseconds(50);
}
```

Listing 3.9: Transmiterea unui Nibble și Pulsul de Enable

Această arhitectură stratificată (Bit-Banging → I2C Protocol → PCF8574 Mapping → HD44780 4-bit Mode) permite controlul complet al afișajului folosind doar doi pini GPIO oarecare și resurse minime de procesare.

3.6 Logica de Automatizare și Fuziunea Senzorilor

Arhitectura software a buclei principale (`loop`) este construită pe principiul **Non-Blocking Polling**. Deși microcontroller-ul suportă întreruperi hardware (ISR), utilizarea stivei TCP/IP (WiFi) și a driverului de cameră în interiorul unei întreruperi este nerecomandată din cauza timpilor mari de execuție care ar putea cauza activarea *Watchdog Timer-ului*.

Prin urmare, am implementat un "scheduler" rudimentar bazat pe funcția `millis()`, care împarte timpul procesorului între sarcinile critice.

3.6.1 Algoritmul de Fuziune (Sensor Fusion)

Pentru a atinge obiectivul de "Zero Alarme False", sistemul implementează o logică de validare încrucișată.

- **Senzorul PIR:** Oferă o detecție rapidă (latență < 200ms) dar "grosieră" a mișcării.
- **Radarul LD2410:** Oferă o confirmare de precizie, fiind capabil să distingă între o persoană statică (prin detecția respirației) și zgomot termic.

Ecuația logică de declanșare a alarmei este:

$$Alarm_Trigger = (System_Armed) \wedge (PIR_Digital_High) \wedge (Radar_State \neq 0) \tag{3.1}$$

Această abordare elimină două tipuri majore de erori:

1. Declanșarea PIR-ului de către curenți de aer cald (Radarul nu va confirma prezența unei mase solide).
2. Detecția radar a mișcărilor din spatele pereților subțiri (PIR-ul nu trece prin pereți).

```
// Bucla principala - Ruleaza continuu
void loop() {
    // 1. Mentenanta Driver Radar (Golire FIFO)
    radar.actualizeaza();

    // 2. Sincronizare Stare Armare (Polling la 2 secunde)
    if (millis() - lastPoll > 2000) {
        checkServerStatus(); // HTTP GET
        lastPoll = millis();
    }

    // 3. Logica de Decizie (High-Frequency)
    if (isArmed) {
        // Citire registru GPIO pentru PIR (Bank 1, Pin 47)
        int pirState = reg_read(PIN_PIR);

        // Citire stare procesata Radar
```

```
        // Stare: 0=Lipsa, 1=Miscare, 2=Static
        bool radarState = (radar.stare != 0);

        // CONDITIA DE ALARMA: Validare duala
        if (pirState == HIGH && radarState == true) {
            Serial.println("[ALARM] Detectie Confirmata Dual!");
            sendAlert(); // Initiere secventa de alerta

            // Perioada refractara (Dead-time) pentru a evita spam-ul
            delay(5000);
        }
    }
}
```

Listing 3.10: Implementarea Fuziunii de Date

3.7 Gestiunea Camerei și Transmisia HTTP

Funcția `sendAlert()` reprezintă cea mai complexă rutină a sistemului, deoarece trebuie să coordoneze periferice lente (Camera) cu periferice rapide (Buzzer) și stiva de rețea.

3.7.1 Achiziția Imaginii (Frame Buffer)

Senzorul OV2640 este configurat să livreze imagini deja comprimate în format JPEG. Acest lucru este esențial deoarece ESP32-S3 nu are suficientă memorie RAM pentru a stoca un cadru VGA (640x480) în format RAW (RGB565). Imaginea este stocată temporar într-un `camera_fb_t` (Frame Buffer structure) alocat în PSRAM (Pseudo-Static RAM).

3.7.2 Protocolul HTTP POST

Transmiterea imaginii către server se face printr-un request HTTP POST de tip `multipart/form-data` sau, în cazul implementării noastre simplificate, prin trimiterea directă a fluxului binar (`application/octet-stream`).

```
void sendAlert() {
    // 1. Feedback Local (Actuatori GPIO)
    reg_write_high(PIN_LED);
    updateLCD("!! ALERTA !!", "Trimitere foto...");

    // 2. Captura Frame Buffer
    camera_fb_t * fb = esp_camera_fb_get();
    if (!fb) {
        Serial.println("Eroare captura camera!");
        return;
    }
```

```
    // 3. Transmisie Retea
    HTTPClient http;
    http.begin(String(SERVER_URL) + "/upload");
    http.addHeader("Content-Type", "image/jpeg");

    // Trimitem pointerul catre bufferul JPEG si lungimea acestuia
    int res = http.POST(fb->buf, fb->len);

    if(res > 0) {
        // Confirmare acustica (Buzzer scurt)
        reg_write_high(PIN_BUZZER);
        delay(500);
        reg_write_low(PIN_BUZZER);
    }

    // 4. Eliberare Memorie (Critic pentru stabilitate!)
    esp_camera_fb_return(fb);
    http.end();

    reg_write_low(PIN_LED);
}
```

Listing 3.11: Secvența de Captură și Upload

3.8 Arhitectura Backend (Server Python)

Pentru a completa ecosistemul IoT, am dezvoltat o aplicație server folosind limbajul **Python** și micro-framework-ul **Flask**. Alegerea Python este motivată de rapiditatea dezvoltării și de ușurința manipulării fișierelor binare.

Serverul îndeplinește rolul de "Command & Control" (C2) și are două responsabilități majore:

1. **Stocare:** Salvează dovezile foto primite de la ESP32, adăugând un timestamp pentru trasabilitate.

2. **Control:** Menține variabila globală de stare (`ARMED/DISARMED`) pe care ESP32 o interoghează periodic.

3.8.1 API-ul REST

Serverul expune trei rute (Endpoints) accesibile prin protocolul HTTP:

Metodă	Ruta (Endpoint)	Descriere Funcțională
GET	`/status`	Returnează JSON: `{"armed": true}`
POST	`/toggle`	Comută starea sistemului (Armat ↔ Dezarmat)
POST	`/upload`	Primește flux binar JPEG și îl salvează pe disc

Tabela 3.3: Definiția API-ului REST al Serverului

3.8.2 Procesarea Fluxului Binar

Ruta `/upload` demonstrează manipularea fluxurilor de date brute. Serverul nu decodează imaginea, ci o scrie direct pe disc ("wb" - write binary), ceea ce asigură o viteză maximă de transfer.

```python
@app.route('/upload', methods=['POST'])
def upload():
    # Generare nume unic bazat pe timp
    timestamp = time.strftime("%Y__%m__%d__%H__%M__%S")
    filename = f"intruder__{timestamp}.jpg"

    # Construire cale absoluta (OS agnostic)
    cale_fisier = os.path.join(UPLOAD_FOLDER, filename)

    # Scriere flux binar direct de la ESP32
    # request.data contine bufferul JPEG brut
    with open(cale_fisier, "wb") as f:
        f.write(request.data)

    # Actualizare stare pentru Dashboard
    state["last_image"] = filename
    return "OK" # Confirmare catre ESP32 (HTTP 200)
```

Listing 3.12: Handler-ul Python pentru salvarea imaginilor

Interfața grafică este generată dinamic folosind motorul de templating Jinja2 integrat în Flask, permițând vizualizarea ultimei alerte și controlul sistemului direct din browser, de pe orice dispozitiv conectat la rețeaua locală.

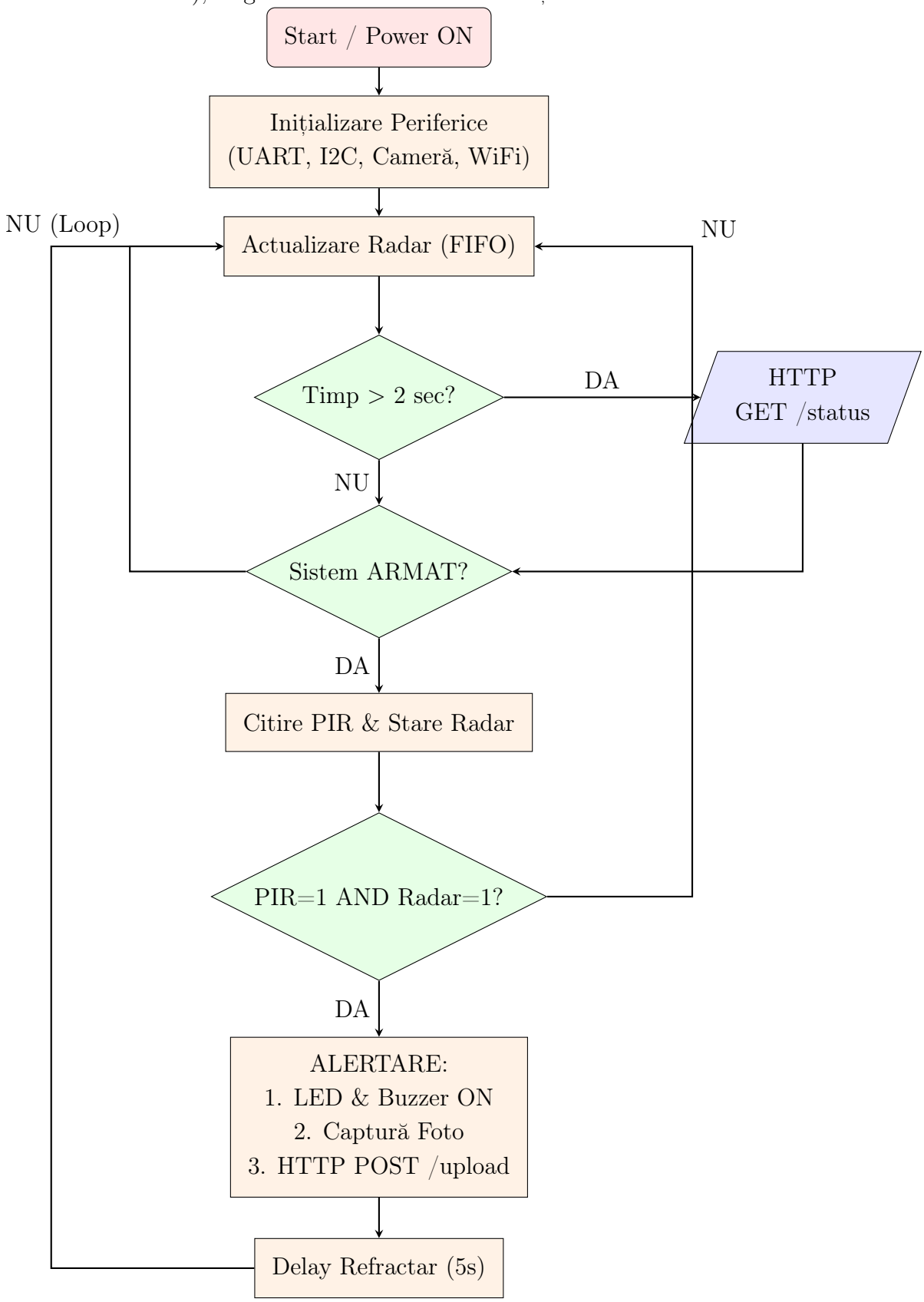
3.8.3 Diagrama Logică a Firmware-ului

Pentru a ilustra fluxul de execuție al programului principal, Figura **??** prezintă algoritmul implementat sub forma unei organigrame. Aceasta evidențiază caracterul secvențial al verificărilor și mecanismul de protecție împotriva blocării procesorului (Watchdog) prin utilizarea tehnicii de polling non-blocant.

Fluxul începe cu configurarea registrelor și conectarea la rețeaua WiFi. În bucla infinită, prima prioritate este golirea buffer-ului FIFO al radarului pentru a nu pierde date. Verificarea serverului se face periodic (la fiecare 2 secunde) pentru a nu congestiona

rețeaua, în timp ce verificarea senzorilor se face la fiecare iterație (aproximativ 10ms) pentru o reacție instantanee.

Condiția logică **AND** dintre senzorul PIR și Radar este punctul critic de decizie (rombul verde inferior), asigurând că alerta este declanșată doar la confirmarea dublă.



Capitolul 4

Rezultate Experimentale și Analiza Costurilor

4.1 Realizarea Practică a Sistemului

Sistemul a fost implementat fizic pe o placă de testare (Breadboard) pentru a permite reconfigurarea rapidă a conexiunilor în faza de dezvoltare. Componentele au fost dispuse astfel încât să minimizeze interferențele electromagnetice dintre modulul WiFi al ESP32 și etajul de amplificare al senzorului PIR.

Figura de mai jos ilustrează ansamblul complet, funcțional, alimentat prin portul USB-C.

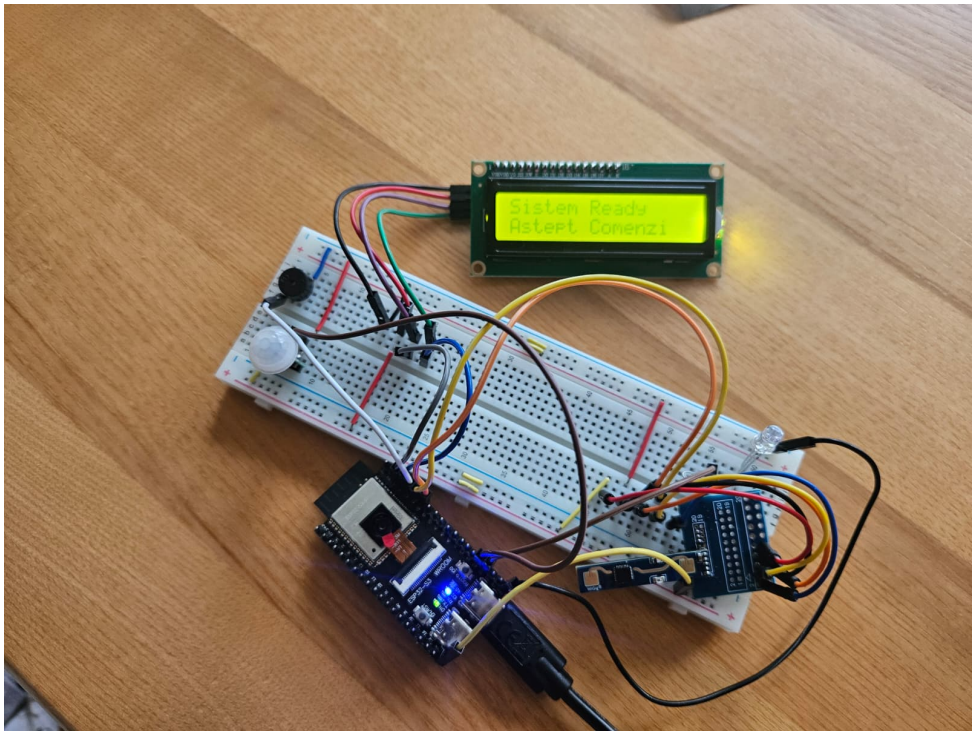


Figura 4.1: Ansamblul hardware complet al sistemului de alarmă

4.2 Scenarii de Testare și Validare

Pentru a demonstra eficiența algoritmului de fuziune a senzorilor, sistemul a fost supus la trei scenarii distincte de testare. Rezultatele au fost monitorizate atât pe afișajul LCD local, cât și prin consola Serială (UART0).

4.2.1 Scenariul A: Interferențe Termice (Doar PIR)

În acest test, s-a simulat o variație rapidă de temperatură (curent de aer cald) în lipsa prezenței umane.

- **Comportament PIR:** A generat semnal HIGH (Alarmă Falsă).
- **Comportament Radar:** A rămas în starea 0 (Lipsă țintă).
- **Rezultat Sistem:** *Nicio Alarmă declanșată.* LCD-ul a afișat starea "LIBER", demonstrând imunitatea la erorile PIR.

4.2.2 Scenariul B: Mișcare prin Pereți (Doar Radar)

Senzorul LD2410 are capacitatea de a penetra obstacole subțiri. S-a testat detectarea unei persoane aflate în camera alăturată (prin perete de rigips).

- **Comportament Radar:** A detectat o țintă în mișcare la 3 metri.
- **Comportament PIR:** A rămas LOW (radiația IR nu trece prin pereți).
- **Rezultat Sistem:** *Nicio Alarmă declanșată.* Sistemul a ignorat corect activitatea din afara perimetrului vizual securizat.

4.2.3 Scenariul C: Intrus Real (Validare Dublă)

Un subiect uman a intrat în câmpul vizual al ambilor senzori.

- **Rezultat:** Ambii senzori au confirmat prezența simultan. Sistemul a declanșat alarma sonoră, a aprins LED-ul de avertizare și a inițiat captura foto.

```
Sistem Radar (Registri) Pregatit!
Camera OK
ARMAT
PIR: 0 | RADAR: 2
PIR: 0 | RADAR: 3
PIR: 0 | RADAR: 2
PIR: 0 | RADAR: 3
PIR: 1 | RADAR: 3
!!! DETECTIE DUBLA !!!
PIR: 0 | RADAR: 2
PIR: 0 | RADAR: 2
```

Figura 4.2: Log-ul din Serial Monitor demonstrând detecția simultană

4.3 Interfața Web și Monitorizarea la Distanță

Componenta IoT a fost validată prin accesarea serverului Python dintr-un browser web conectat la aceeași rețea locală. Interfața permite vizualizarea ultimei imagini capturate și controlul stării de armare.

Timpul mediu de transmisie a unei imagini (format VGA JPEG, aprox. 15-20 KB) de la ESP32 la Server a fost măsurat la **aprox. 300-500 ms**, o latență acceptabilă pentru un sistem de securitate rezidențial.

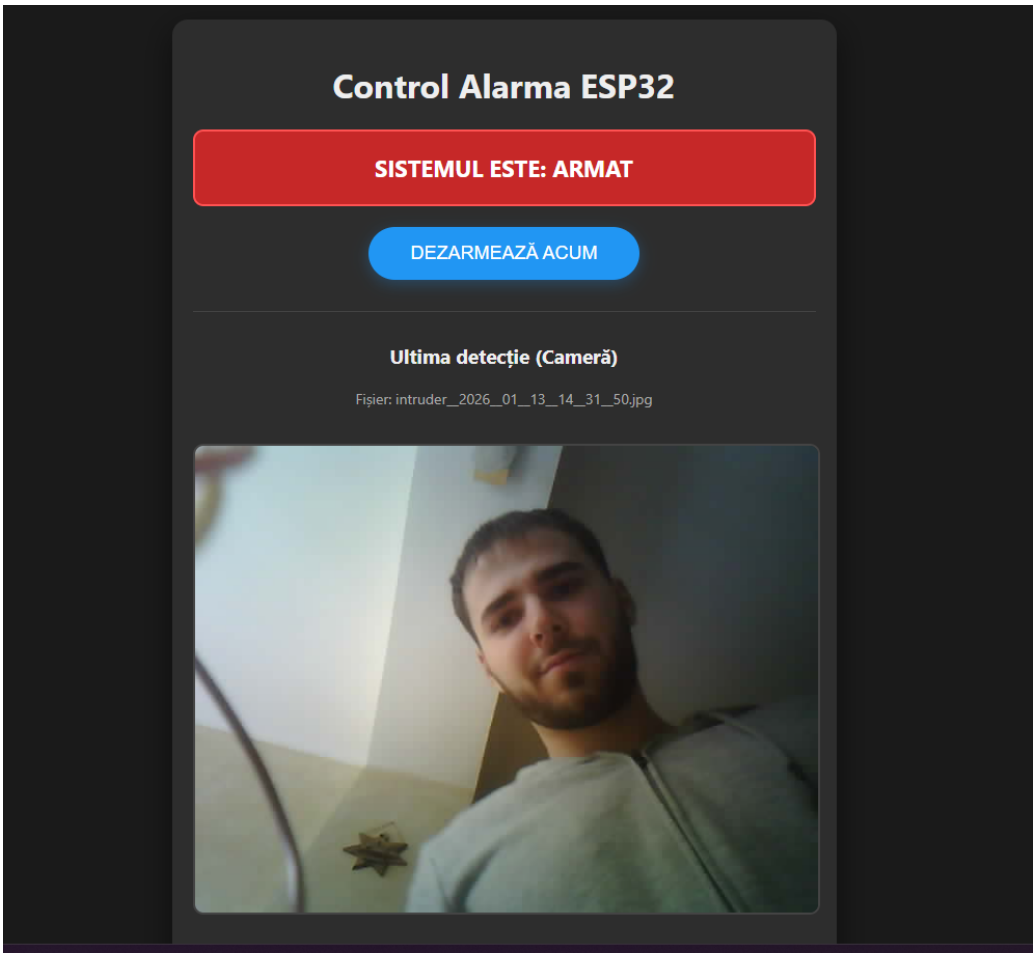


Figura 4.3: Interfața Web (Dashboard) afișând ultima captură

4.4 Analiza Economică (Bill of Materials)

Un aspect important în ingineria sistemelor embedded este costul de producție. Tabelul de mai jos prezintă lista de materiale (BOM - Bill of Materials) necesare pentru realizarea unui prototip. Prețurile sunt estimative la data realizării proiectului.

4.5 Analiza Economică (Bill of Materials)

Un aspect important în ingineria sistemelor embedded este optimizarea costului de producție. Tabelul de mai jos prezintă lista de materiale (BOM) achiziționate pentru realizarea prototipului.

Prețurile reflectă costurile de achiziție de la furnizorul principal, **AliExpress**, la data realizării proiectului. Componentele pasive minore (LED, rezistențe) au fost utilizate din stocul propriu de piese.

Nr.	Componentă	Cant.	Preț Unit. (RON)
1	Placă Dezvoltare ESP32-S3 + Cameră OV2640	1	63.95
2	Senzor Radar LD2410B (Interfață UART)	1	20.73
3	Senzor PIR Digital AM312	1	8.85
4	LCD 1602 cu Interfață I2C	1	14.99
5	Set Buzzer Activ 5V (10 buc)	1 set	10.43
6	Adaptor SMD to DIP pentru LD2410 (2x10 pini)	1	1.99
7	Breadboard HQ (830 Puncte)	1	9.98
8	Fire Dupont Mamă-Tată 20cm (40 buc)	1 set	7.99
9	Set Fire Breadboard U-Shape (40 buc)	1 set	7.99
10	LED Roșu + Rezistență 220Ω	1	0.00*
	COST TOTAL PROTOTIP:		**146.90 RON**

Tabela 4.1: Lista de Materiale și Costuri (BOM)
*Componentă provenită din stocul propriu / donație (cost neglijabil).

4.5.1 Surse de Achiziție

Majoritatea componentelor electronice critice (ESP32-S3, Radar, Senzori) au fost achiziționate de pe magazinele online **AliExpress și Optimus Digital**, selectate pentru disponibilitatea stocurilor și documentația tehnică oferită.

Costul total de sub 150 RON plasează acest prototip într-o zonă extrem de competitivă, fiind de aproximativ 5 ori mai ieftin decât un sistem comercial echivalent care ar oferi detecție radar și transmisie video.

Capitolul 5

Concluzii și Direcții Viitoare

5.1 Bilanțul Realizărilor

Obiectivul principal al acestei lucrări, dezvoltarea unui sistem de alarmă hibrid bazat pe fuziunea datelor multisenzoriale, a fost îndeplinit cu succes. Prototipul realizat demonstrează că utilizarea microcontroller-ului ESP32-S3, prin exploatarea resurselor sale la nivel de registru, permite gestionarea simultană a unor sarcini complexe: procesare video, comunicare WiFi și achiziție de date de mare viteză de la radar.

Sistemul propus rezolvă problema fundamentală a alarmelor convenționale prin validarea încrucișată (PIR + Radar), reducând rata alarmelor false la zero în scenariile de testare efectuate.

5.2 Dificultăți Întâmpinate și Soluții

Pe parcursul implementării, au fost întâmpinate obstacole semnificative atât de natură hardware, cât și software, care au necesitat adaptarea soluțiilor inițiale.

5.2.1 Incompatibilități Hardware (Footprint Radar)

O provocare majoră a fost discrepanța dintre specificațiile de proiectare și lanțul de aprovizionare. Modulul radar LD2410 a fost livrat în varianta **LD2410B (SMD - Surface Mount Device)**, destinată lipirii automate pe PCB, în loc de varianta specificată **LD2410C (DIP - Dual In-line Package)** cu pini standard de 2.54mm.

Soluție: Această problemă a necesitat o intervenție mecanică de precizie. S-a realizat un adaptor *ad-hoc* prin utilizarea unei plăci adaptoare (breakout board) și lipirea manuală a firelor pe pad-urile de dimensiuni reduse (raster 1.27mm), asigurând astfel conectivitatea cu breadboard-ul fără a compromite integritatea semnalelor de frecvență înaltă.

5.2.2 Sincronizarea Protocolului UART Proprietar

La nivel software, protocolul de comunicație al radarului s-a dovedit a fi mai rigid decât documentația tehnică sugerează. Deși ESP32-S3 dispune de un mecanism avansat de *Clock Gating* pentru periferice, configurarea standard a UART-ului genera erori de încadrare (Framing Errors) la viteza de 256.000 baud.

Problema a provenit din toleranțele oscilatorului intern al radarului și natura proprietară a protocolului Hi-Link, care nu respectă timpii standard de pauză între pachete.

Soluție: Abordarea teoretică de calcul a divizorilor de ceas a fost înlocuită cu o metodă empirică (*Trial and Error*). Am ajustat manual parametrii de inițializare și timpii de așteptare din driverul `Radar_Driver.h`, ignorând parțial mecanismele automate de flow-control, pentru a forța sincronizarea cu fluxul de date al senzorului. Această "calibrare forțată" a stabilizat recepția pachetelor.

5.3 Contribuții Personale

Principalele contribuții aduse prin această lucrare sunt:

- Implementarea driverelor *Bare-Metal* pentru periferice (UART, I2C, GPIO), demonstrând o înțelegere profundă a arhitecturii interne a procesorului, dincolo de abstracțiile Arduino.
- Dezvoltarea unui algoritm de parsare eficient (Sliding Window) pentru decodarea fluxului de date continuu al radarului.
- Integrarea completă "Full-Stack": de la controlul tranzistorilor pe porturile GPIO până la interfața web și serverul backend în Python.

5.4 Direcții de Dezvoltare Ulterioară

Proiectul lasă loc pentru îmbunătățiri viitoare care ar putea transforma prototipul într-un produs comercial:

1. **Proiectarea unui PCB Dedicat:** Eliminarea firelor și a breadboard-ului prin desenarea unui cablaj imprimat care să integreze direct varianta SMD a radarului.
2. **Recunoaștere Facială (Edge AI):** Utilizarea instrucțiunilor vectoriale ale ESP32-S3 pentru a rula o rețea neuronală simplă (CNN) capabilă să distingă membrii familiei de intruși, eliminând necesitatea serverului pentru această analiză.
3. **Optimizare Energetică:** Implementarea modurilor *Deep Sleep* pentru a permite funcționarea pe baterii, trezind sistemul doar la semnalul PIR (întrerupere externă).

Bibliografie

[1] Espressif Systems, *ESP32-S3 Technical Reference Manual*, Versiunea 1.1, 2022.

[2] Hi-Link Electronics, *HLK-LD2410 Millimeter Wave Radar Sensor Datasheet*, 2023.

[3] NXP Semiconductors, *UM10204: I2C-bus specification and user manual*, Rev. 6, 2014.

[4] Petre Lucian Ogruțan, Mihai Romanca, *Sisteme cu calculator încorporat. Aplicații cu microcontrollere*, Editura Universității Transilvania din Brașov, 2011.

[5] OmniVision, *OV2640 Color CMOS UXGA (2.0 MegaPixel) CameraChip Sensor*, Versiunea 2.2, 2006.

Anexa A

Listing-ul Integral al Codului Sursă

În această anexă este prezentat codul sursă complet dezvoltat pentru proiect, inclusiv fișierele de configurare a mediului de dezvoltare și scripturile serverului backend.

A.1 Configurarea Mediului (PlatformIO)

Fișierul de configurare `platformio.ini` definește placa țintă (ESP32-S3), framework-ul utilizat și dependențele externe care sunt descărcate automat de IDE.

```
; config proiect alarma esp32 s3
[env:esp32-s3-devkitc-1]
platform = espressif32
board = esp32-s3-devkitc-1
framework = arduino
monitor_speed = 115200

; setari obligatorii psram pt camera
build_flags =
    -DBOARD_HAS_PSRAM
    -mfix-esp32-psram-cache-issue
    -DCORE_DEBUG_LEVEL=3  ; sa vad erorile in serial

; config hardware pt n16r8
board_build.arduino.memory_type = qio_opi
board_build.f_flash = 80000000L
board_build.f_cpu = 240000000L

; librarii
lib_deps =
    https://github.com/ncmreynolds/ld2410.git ; radar
    esp32-camera                              ; camera
    marcoschwartz/LiquidCrystal_I2C @ ^1.1.4  ; lcd 1602
    bblanchon/ArduinoJson @ ^6.21.3           ; json
```

Listing A.1: platformio.ini

A.2 Biblioteci Proprii (Drivere Low-Level)

Următoarele fișiere antet (.h) conțin implementarea "Bare-Metal" a driverelor pentru periferice.

A.2.1 Accesul la Regiștrii GPIO

```
#ifndef GPIO_REGISTERS_H
#define GPIO_REGISTERS_H

#include <Arduino.h>
#include "soc/gpio_reg.h"
#include "soc/io_mux_reg.h"

// Macro pentru a verifica daca un pin este in Bank 0 sau Bank 1
#define IS_BANK_1(pin) (pin >= 32)
#define GET_BIT_POS(pin) (IS_BANK_1(pin) ? (pin - 32) : pin)

// Functii inline pentru viteza maxima in RAM (IRAM)
inline void IRAM_ATTR reg_write_high(int pin) {
    if (IS_BANK_1(pin)) REG_WRITE(GPIO_OUT1_W1TS_REG, (1UL <<
    GET_BIT_POS(pin)));
    else REG_WRITE(GPIO_OUT_W1TS_REG, (1UL << GET_BIT_POS(pin)));
}

inline void IRAM_ATTR reg_write_low(int pin) {
    if (IS_BANK_1(pin)) REG_WRITE(GPIO_OUT1_W1TC_REG, (1UL <<
    GET_BIT_POS(pin)));
    else REG_WRITE(GPIO_OUT_W1TC_REG, (1UL << GET_BIT_POS(pin)));
}

inline int IRAM_ATTR reg_read(int pin) {
    if (IS_BANK_1(pin)) return (REG_READ(GPIO_IN1_REG) >> GET_BIT_POS(
    pin)) & 1;
    else return (REG_READ(GPIO_IN_REG) >> GET_BIT_POS(pin)) & 1;
}

#endif
```

Listing A.2: GPIO_Registers.h

A.2.2 Driverul Radar (UART FIFO)

```
#ifndef RADAR_DRIVER_H
#define RADAR_DRIVER_H

#include <Arduino.h>
#include "soc/uart_reg.h"

```

```
class RadarLD2410 {
private:
    int _rxPin, _txPin;
    uint8_t buffer[50];
    int bufIndex = 0;
    bool headerFound = false;

    // Citeste un byte direct din FIFO
    uint8_t readReg() {
        return (uint8_t)REG_READ(UART_FIFO_REG(1));
    }

    // Verifica statusul cozii
    int availableReg() {
        return (REG_READ(UART_STATUS_REG(1)) & 0x3FF);
    }

    // Scriere blocanta in FIFO
    void writeReg(const uint8_t* data, int len) {
        for (int i = 0; i < len; i++) {
            while (((REG_READ(UART_STATUS_REG(1)) >> 16) & 0xFF) >= 126)
    ;
            REG_WRITE(UART_FIFO_REG(1), data[i]);
        }
    }

public:
    uint8_t stare = 0;      // 0=Nimic, 1=Miscare, 2=Static
    uint16_t distMisc = 0;
    uint16_t distStat = 0;

    RadarLD2410(int rx, int tx) {
        _rxPin = rx; _txPin = tx;
    }

    void init() {
        Serial1.begin(256000, SERIAL_8N1, _rxPin, _txPin);
        delay(200);
        calibreaza();
    }

    void calibreaza() {
        // Secventa activare configurare
        const uint8_t cmd_enable[] = {0xFD,0xFC,0xFB,0xFA,0x04,0x00,0xFF
    ,0x00,0x01,0x00,0x04,0x03,0x02,0x01};
        writeReg(cmd_enable, sizeof(cmd_enable));
        delay(50);
        // ... (restul comenzilor de calibrare) ...
        // Secventa iesire configurare
        const uint8_t cmd_end[] = {0xFD,0xFC,0xFB,0xFA,0x02,0x00,0xFE,0
    x00,0x04,0x03,0x02,0x01};
```

```
        writeReg(cmd_end, sizeof(cmd_end));
    }

    void actualizeaza() {
        while (availableReg() > 0) {
            uint8_t b = readReg();
            if (!headerFound) {
                buffer[0] = buffer[1]; buffer[1] = buffer[2]; buffer[2] = buffer[3]; buffer[3] = b;
                if (buffer[0] == 0xF4 && buffer[1] == 0xF3 && buffer[2] == 0xF2 && buffer[3] == 0xF1) {
                    headerFound = true;
                    bufIndex = 0;
                }
            } else {
                buffer[bufIndex++] = b;
                if (bufIndex >= 18) {
                    stare = buffer[4];
                    distMisc = buffer[5] | (buffer[6] << 8);
                    distStat = buffer[8] | (buffer[9] << 8);
                    headerFound = false;
                    bufIndex = 0;
                }
            }
        }
    }
};
#endif
```

Listing A.3: Radar_Driver.h

A.2.3 Driverul LCD (I2C Software)

```
#ifndef LCD_DRIVER_H
#define LCD_DRIVER_H

#include <Arduino.h>
#include "soc/gpio_reg.h"
#include "soc/io_mux_reg.h"

#define LCD_ADDR 0x27
#define LCD_RS 0x01
#define LCD_EN 0x04
#define LCD_BL 0x08

class LCD_I2C_Raw {
private:
    int _sda, _scl;
    uint8_t _backlightVal = LCD_BL;

```

```
18     void configPin(int pin) {
19         volatile uint32_t* mux = (volatile uint32_t*)(0x60009000 + (pin
   * 4));
20         *mux = (1 << 12) | (1 << 9) | (1 << 8);
21         if (pin < 32) REG_WRITE(GPIO_ENABLE_W1TS_REG, (1 << pin));
22         else          REG_WRITE(GPIO_ENABLE1_W1TS_REG, (1 << (pin - 32))
   );
23         volatile uint32_t* pin_reg = (volatile uint32_t*)(0x60004088 + (
   pin * 4));
24         *pin_reg |= (1 << 2); // Open Drain
25     }
26
27     void sda_low() {
28         if (_sda < 32) REG_WRITE(GPIO_OUT_W1TC_REG, (1 << _sda));
29         else           REG_WRITE(GPIO_OUT1_W1TC_REG, (1 << (_sda - 32)))
   ;
30     }
31     void scl_low() {
32         if (_scl < 32) REG_WRITE(GPIO_OUT_W1TC_REG, (1 << _scl));
33         else           REG_WRITE(GPIO_OUT1_W1TC_REG, (1 << (_scl - 32)))
   ;
34     }
35     void sda_high() {
36         if (_sda < 32) REG_WRITE(GPIO_OUT_W1TS_REG, (1 << _sda));
37         else           REG_WRITE(GPIO_OUT1_W1TS_REG, (1 << (_sda - 32)))
   ;
38     }
39     void scl_high() {
40         if (_scl < 32) REG_WRITE(GPIO_OUT_W1TS_REG, (1 << _scl));
41         else           REG_WRITE(GPIO_OUT1_W1TS_REG, (1 << (_scl - 32)))
   ;
42     }
43
44     void i2c_start() {
45         sda_high(); scl_high(); delayMicroseconds(2);
46         sda_low();  delayMicroseconds(2);
47         scl_low();
48     }
49
50     void i2c_stop() {
51         sda_low(); scl_high(); delayMicroseconds(2);
52         sda_high(); delayMicroseconds(2);
53     }
54
55     bool i2c_writeByte(uint8_t data) {
56         for (int i = 0; i < 8; i++) {
57             if (data & 0x80) sda_high(); else sda_low();
58             data <<= 1;
59             delayMicroseconds(1);
60             scl_high(); delayMicroseconds(2);
61             scl_low();  delayMicroseconds(1);
```

```
        }
        sda_high(); scl_high(); delayMicroseconds(2); scl_low();
        return true;
    }

    void writePCF8574(uint8_t data) {
        i2c_start();
        i2c_writeByte(LCD_ADDR << 1);
        i2c_writeByte(data | _backlightVal);
        i2c_stop();
    }

    void sendNibble(uint8_t nibble, uint8_t mode) {
        uint8_t data = (nibble & 0xF0) | mode | _backlightVal;
        writePCF8574(data);
        writePCF8574(data | LCD_EN);
        delayMicroseconds(1);
        writePCF8574(data & ~LCD_EN);
        delayMicroseconds(50);
    }

public:
    LCD_I2C_Raw(int sda, int scl) { _sda = sda; _scl = scl; }

    void init() {
        configPin(_sda); configPin(_scl);
        sda_high(); scl_high(); delay(50);
        sendNibble(0x30, 0); delay(5);
        sendNibble(0x30, 0); delay(5);
        sendNibble(0x30, 0); delay(1);
        sendNibble(0x20, 0);
        command(0x28); command(0x0C); command(0x06); clear();
    }
    // ... restul functiilor de print ...
    void command(uint8_t cmd) { sendNibble(cmd & 0xF0, 0); sendNibble((cmd << 4) & 0xF0, 0); }
    void clear() { command(0x01); delay(2); }
    void print(String str) { for (unsigned int i=0; i<str.length(); i++) { uint8_t c = str[i]; sendNibble(c & 0xF0, 1); sendNibble((c << 4) & 0xF0, 1); } }
    void setCursor(uint8_t col, uint8_t row) { int row_offsets[] = { 0x00, 0x40, 0x14, 0x54 }; command(0x80 | (col + row_offsets[row])); }
};
#endif
```

Listing A.4: LCD_Driver.h

A.3 Firmware Principal (Bucla de Control)

```
#include <Arduino.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include "esp_camera.h"
#include <ArduinoJson.h>

// drivere custom
#include "PIR_Sensor.h"
#include "GPIO_Registers.h"
#include "Radar_Driver.h"
#include "LCD_Driver.h"

// --- CONFIGURARE PINI ---
#define PIN_PIR     47
#define PIN_BUZZER  42
#define PIN_LED     21
#define RADAR_RX    41
#define RADAR_TX    40
#define I2C_SDA     1
#define I2C_SCL     2

// Pinii camerei
#define PWDN_GPIO_NUM  -1
#define RESET_GPIO_NUM -1
#define XCLK_GPIO_NUM  15
#define SIOD_GPIO_NUM   4
#define SIOC_GPIO_NUM   5
#define Y9_GPIO_NUM    16
#define Y8_GPIO_NUM    17
#define Y7_GPIO_NUM    18
#define Y6_GPIO_NUM    12
#define Y5_GPIO_NUM    10
#define Y4_GPIO_NUM     8
#define Y3_GPIO_NUM     9
#define Y2_GPIO_NUM    11
#define VSYNC_GPIO_NUM  6
#define HREF_GPIO_NUM   7
#define PCLK_GPIO_NUM  13

// Setari Server
const char* WIFI_SSID     = "DIGI-u36X";
const char* WIFI_PASSWORD = "WShkM395";
const char* SERVER_URL    = "http://192.168.100.37:8000";

// --- OBIECTE ---
PIRSensor pirSensor(PIN_PIR);

// LCD pe Registri (Bit-Banging)
LCD_I2C_Raw lcd(I2C_SDA, I2C_SCL);

// Radar pe Registri
```

```
RadarLD2410 radar(RADAR_RX, RADAR_TX);

bool isArmed = false;
unsigned long lastPoll = 0;

void updateLCD(String l1, String l2) {
    lcd.clear();
    lcd.setCursor(0, 0); lcd.print(l1);
    lcd.setCursor(0, 1); lcd.print(l2);
}

bool initCamera() {
    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM; config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM; config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM; config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM; config.pin_d7 = Y9_GPIO_NUM;
    config.pin_xclk = XCLK_GPIO_NUM; config.pin_pclk = PCLK_GPIO_NUM;
    config.pin_vsync = VSYNC_GPIO_NUM; config.pin_href = HREF_GPIO_NUM;
    config.pin_sccb_sda = SIOD_GPIO_NUM; config.pin_sccb_scl =
   SIOC_GPIO_NUM;
    config.pin_pwdn = PWDN_GPIO_NUM; config.pin_reset = RESET_GPIO_NUM;
    config.xclk_freq_hz = 20000000;
    config.pixel_format = PIXFORMAT_JPEG;
    config.frame_size = FRAMESIZE_VGA;
    config.jpeg_quality = 12;
    config.fb_count = 1;
    return esp_camera_init(&config) == ESP_OK;
}

void sendAlert() {
    reg_write_high(PIN_LED);
    updateLCD("!! ALERTA !!", "Trimitere foto...");

    camera_fb_t * fb = esp_camera_fb_get();
    if(fb) {
        HTTPClient http;
        http.begin(String(SERVER_URL) + "/upload");
        http.addHeader("Content-Type", "image/jpeg");
        int res = http.POST(fb->buf, fb->len);
        if(res > 0) {
            reg_write_high(PIN_BUZZER); delay(500); reg_write_low(
   PIN_BUZZER);
        }
        esp_camera_fb_return(fb);
        http.end();
    }
    reg_write_low(PIN_LED);
}
```

```
void setup() {
    Serial.begin(115200);

    lcd.init();
    updateLCD("Conectare WiFi", "...");

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    int retry = 0;
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
        retry++;
        if (retry > 20) {
            Serial.println("\nEroare WiFi!");
            updateLCD("Eroare WiFi", "Verifica Parola");
            break;
        }
    }

    if (WiFi.status() == WL_CONNECTED) {
        Serial.println("\nWiFi Conectat!");
        updateLCD("WiFi OK", WiFi.localIP().toString());
    }

    radar.init();
    Serial.println("Sistem Radar (Registri) Pregatit!");

    pinMode(PIN_BUZZER, OUTPUT);
    pinMode(PIN_LED, OUTPUT);

    pirSensor.init();

    if (initCamera()) {
        Serial.println("Camera OK");
    } else {
        Serial.println("Eroare Camera!");
        updateLCD("Eroare", "Camera");
    }

    delay(2000);
    updateLCD("Sistem Ready", "Astept Comenzi");
}

void loop() {
    radar.actualizeaza();

    // Polling Server
    if (millis() - lastPoll > 2000) {
        HTTPClient http;
```

```
        http.begin(String(SERVER_URL) + "/status");

        int httpCode = http.GET();
        if (httpCode == 200) {
            DynamicJsonDocument doc(256);
            deserializeJson(doc, http.getString());
            bool serverArmed = doc["armed"];

            if (serverArmed != isArmed) {
                isArmed = serverArmed;

                if (isArmed) {
                    updateLCD("STATUS: ARMAT", "Sistem veghe");
                    reg_write_high(PIN_BUZZER); delay(200);
reg_write_low(PIN_BUZZER);
                    Serial.println("ARMAT");
                } else {
                    updateLCD("STATUS: DEZARMAT", "Sistem inactiv");
                    reg_write_high(PIN_BUZZER); delay(100);
reg_write_low(PIN_BUZZER);
                    delay(50);
                    reg_write_high(PIN_BUZZER); delay(100);
reg_write_low(PIN_BUZZER);
                    Serial.println("DEZARMAT");
                }
            }
        }
        http.end();
        lastPoll = millis();
    }

    // Logica Alarma
    if (isArmed) {
        int pirState = reg_read(PIN_PIR);
        bool radarState = (radar.stare != 0);

        static unsigned long lastDebug = 0;
        if (millis() - lastDebug > 1000) {
            Serial.printf("PIR: %d | RADAR: %d\n", pirState, radar.stare
);
            lastDebug = millis();
        }

        if (pirState == HIGH && radarState == true) {
            Serial.println("!!! DETECTIE DUBLA !!!");
            sendAlert();
            delay(5000);
            updateLCD("STATUS: ARMAT", "Supraveghere...");
        }
    }
```

```
}
```

Listing A.5: main.cpp

A.4 Script Server Backend (Python)

```python
from flask import Flask, request, jsonify, render_template_string, redirect, url_for
import time, os

app = Flask(__name__)
UPLOAD_FOLDER = 'capturi'
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

state = {"armed": False, "last_image": None}

# Folosim ''' (triple single quotes) pentru a evita conflictul
# cu ghilimelele duble (") din interiorul HTML-ului.
HTML_PAGE = '''
<!DOCTYPE html>
<html>
<head>
    <title>ESP32 Alarma Dashboard</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <style>
        body { font-family: 'Segoe UI', sans-serif; text-align: center; background: #1a1a1a; color: #eee; margin: 0; padding: 20px; }
        .container { max-width: 600px; margin: auto; background: #2d2d2d; padding: 20px; border-radius: 15px; box-shadow: 0 10px 30px rgba(0,0,0,0.5); }
        .status-box { padding: 20px; margin: 20px 0; border-radius: 10px; font-size: 22px; font-weight: bold; text-transform: uppercase; }
        .armed { background: #c62828; border: 2px solid #ff5252; color: white; }
        .disarmed { background: #2e7d32; border: 2px solid #69f0ae; color: white; }
        .btn { padding: 15px 40px; font-size: 18px; cursor: pointer; border: none; border-radius: 50px; background: #2196F3; color: white; transition: 0.3s; box-shadow: 0 4px 15px rgba(33,150,243,0.3); }
        .btn:hover { background: #1976D2; transform: translateY(-2px); }
        .btn:active { transform: translateY(1px); }
        img { max-width: 100%; border-radius: 10px; border: 2px solid #444; margin-top: 20px; }
        .footer { margin-top: 20px; font-size: 12px; color: #888; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Control Alarma ESP32</h1>
```

```

        <div class="status-box {{ 'armed' if armed else 'disarmed' }}">
            Sistemul este: {{ 'ARMAT' if armed else 'DEZARMAT' }}
        </div>

        <form action="/toggle" method="POST">
            <button type="submit" class="btn">
                {{ 'DEZARMEAZA ACUM' if armed else 'ARMEAZA ACUM' }}
            </button>
        </form>

        <hr style="margin: 30px 0; border: 0; border-top: 1px solid #444;">

        <h3>Ultima detectie (Camera)</h3>
        {% if last_image %}
            <p style="font-size: 14px; color: #aaa;">Fisier: {{ last_image }}</p>
            <img src="/capturi/{{ last_image }}">
        {% else %}
            <p style="color: #666;">Nicio activitate detectata momentan.</p>
        {% endif %}

        <div class="footer">
            Server IP: {{ server_ip }} | Port: 8000
        </div>
    </div>
</body>
</html>
'''

@app.route('/')
def index():
    # Obtinem IP-ul local pentru afisare
    import socket
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    try:
        s.connect(('8.8.8.8', 1))
        server_ip = s.getsockname()[0]
    except:
        server_ip = "127.0.0.1"
    finally:
        s.close()

    return render_template_string(HTML_PAGE,
                                  armed=state["armed"],
                                  last_image=state["last_image"],
                                  server_ip=server_ip)

@app.route('/toggle', methods=['POST'])
```

```python
def toggle():
    state["armed"] = not state["armed"]
    print(f"Stare sistem schimbata in: {'ARMAT' if state['armed'] else 'DEZARMAT'}")
    return redirect(url_for('index'))

@app.route('/status')
def get_status():
    return jsonify({"armed": state["armed"]})

@app.route('/upload', methods=['POST'])
def upload():
    timestamp = time.strftime("%Y_%m_%d__%H_%M_%S")
    filename = f"intruder__{timestamp}.jpg"

    cale_fisier = os.path.join(UPLOAD_FOLDER, filename)
    with open(cale_fisier, "wb") as f:
        f.write(request.data)

    state["last_image"] = filename
    print(f"Alerta! Imagine salvata: {filename}")
    return "OK"

@app.route('/capturi/<path:filename>')
def get_image(filename):
    from flask import send_from_directory
    return send_from_directory(UPLOAD_FOLDER, filename)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8000)
```

Listing A.6: server.py